

```
#flags );
    printf("Received following data on stream %d\n\n%s\n",
        sndrcvinfo.sinfo_stream, buffer);
}
close(cfd);
return 0;
}
```

The server is sending three messages on three different streams, and the client is just receiving the messages and printing them on the screen (see Figures 1 and 2). The code is similar to the TCP client, as we are again making calls to the same functions. The difference is that we are creating an iterative server, similar to the one for UDP, but we have an *accept()* call here. The client does the reverse, and receives the messages coming from the server to the client. Now let's try to understand the functions that we used:

```
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/scp.h>

int sctp_sendmsg (int sd, const void *msg, size_t len,
    struct sockaddr *to, socklen_t tolen, uint32_t ppid,
    uint32_t
    flags, uint16_t stream_no, uint32_t timetolive, uint32_t
    context);
```

We are using this function to send a message from a socket while using the advanced features of SCTP. The first argument to the function is *sd*, the socket descriptor, from which the message *msg* of length *len* is sent. The fourth argument is to give the destination address—*tolen* specifies the length of the destination address, while *stream_no* identifies the stream number to send this message to. The *flags* parameter is used to send some options to the receiver. You can check out the manual pages for *sctp_sendmsg()*. The *timetolive* parameter is time in milliseconds after which the message will expire if not sent by then; the zero here indicates that no time-out is set. The *context* is the value passed to the upper layer along with the undelivered message, if an error occurs while sending the message. When successful, it will return the number of bytes sent, or -1 on error. Next is the *sctp_recvmsg()* function:

```
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/scp.h>

int sctp_recvmsg(int sd, void *msg, size_t len, struct
    sockaddr *from, socklen_t *fromlen, struct sctp_sndrcvinfo
    *sinfo, int *msg_flags);
```

This function does the reverse of the *sctp_sendmsg* function and is used to receive a message. The parameters are similar. The socket *sd* receives the *msg* of length *len* from the address **from* with a length **fromlen*, and **sinfo* is a pointer to the address, which will be filled upon receipt of the message. *msg_flags* is a pointer to an integer with flags like *MSG_NOTIFICATION* or *MSG_EOR*. It returns the number of bytes received, or -1 on error.

```
#include <sys/types.h>
#include <sys/socket.h>

int setsockopt(int sockfd, int level, int optname,
    const void *optval, socklen_t optlen);
```

This function is used to set the options for the socket *sockfd*. The next argument is the *level* at which the option resides. To manipulate options at the sockets API level, the level is specified as *SOL_SOCKET*. *optname* and any specified options are passed uninterpreted to the appropriate protocol module for interpretation. The *level* and *optname* are defined in *sys/socket.h*. The arguments *optval* and *optlen* are used to access option values for *setsockopt()* that are stored in the structure. The options we set in the server are:

```
initmsg.sinit_num_ostreams = 3;
initmsg.sinit_max_instreams = 3;
initmsg.sinit_max_attempts = 2;
```

Here, the first two lines tell us that the output and input streams available are three, and the maximum attempts will be two. The same options are set in the client program. Other options are set for the events. This structure will be filled when an event like 'message received' occurs, and our program is notified. Its counterpart function is *getsockopts()* (look up the *man* pages for help). The rest of the code is simple to understand.

Now compile and run the program; make sure you have installed *sctp-tools* so that you'll have *sctp.h* at */netinet/*. To compile, use *gcc sctpserver.c -lsctp -o server* && *gcc sctpcient.c -lsctp -o client*—and to run, use the following code:

```
$ ./server &
$ ./client
```

I'm wrapping up the series for now, but will keep adding more to the topic, on and off. Mail me in case you need any help. See you guys; LFY is awesome... FOSS rocks! 

By: Pankaj Tanwar a.k.a. singularity

The author is a FOSS enthusiast, loves rock music and C, and can be reached at pankaj.tux@gmail.com.